Все задания для 15 варианта

Лабораторная работа 4. Подготовить 10 осмысленных запросов, демонстрирующих изменения структуры БД (из лабораторной работы 2). Запросы должны включать не менее 3-х таблиц. Обязательны запросы с подзапросами, группировкой и оконными функциями.

Изменения в структуре БД (вариант 15):

2. Возможность блокировки части мест из продажи на рейс.
6.«Бонусная программа» для пассажиров – накопление миль, использование их при оплате билетов.
9. Билеты с багажом и без багажа. Доплата за багаж.

Лабораторная работа 5. Реализовать в БД процедуры (триггеры, если нужно)
1) реализующие работу с изменениями в структуре (из лабораторной работы 2)
2) по заданию для лабораторной работы 5

Задание ЛР5 (вариант 15):

5. Отображения динамики пассажиропотока по разным направлениям.

Лабораторная работа 6. Оптимизация запросов. Подготовить предложения по оптимизации работы трех любых запросов из лабораторных работ 1 и/или 4. Допускается изменение запроса (в том числе добавление «хинтов»), изменение схемы БД, добавление новых объектов (индексы, кластеры, материализованные представления).

Результат предложенного (оптимизация) должен быть продемонстрирован.

Лабораторная работа 7. Реализовать программу для работы с БД, использующую ранее подготовленные процедуры.

Лабораторная работа 8. Доработка струкутры, запросов, процедур и программного обеспечения.

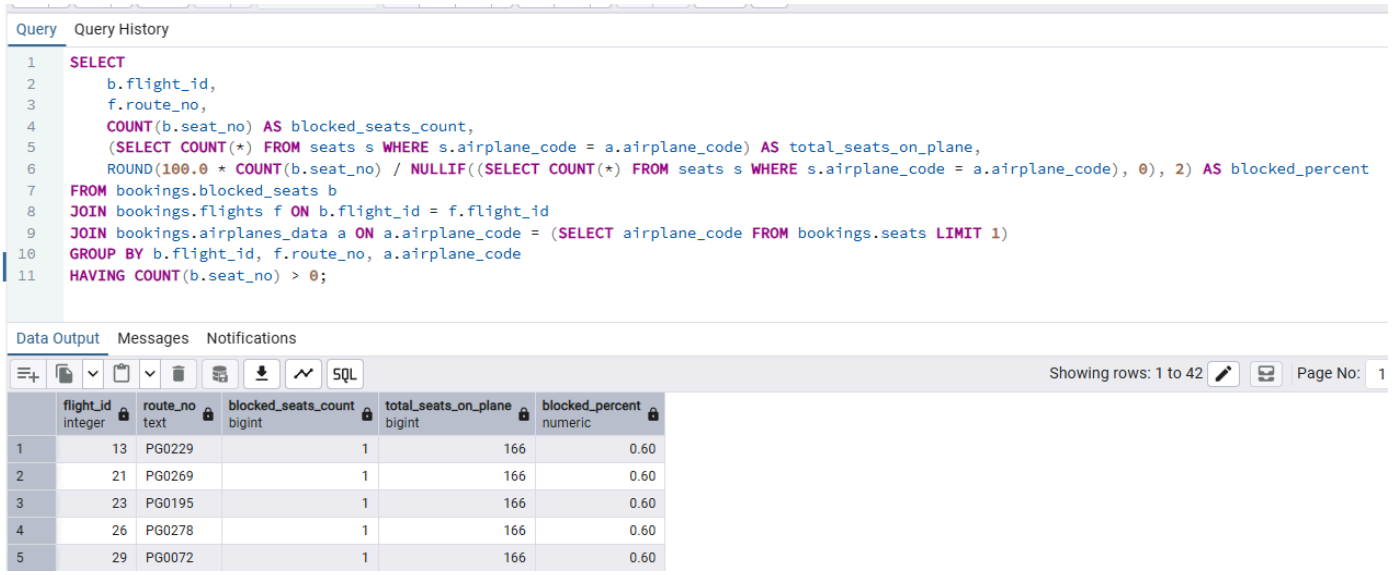
Лабораторная работа 4

Лабораторная работа 4. Подготовить 10 осмысленных запросов, демонстрирующих изменения структуры БД (из лабораторной работы 2). Запросы должны включать не менее 3-х таблиц. Обязательны запросы с подзапросами, группировкой и оконными функциями.

Изменения в структуре БД (вариант 15):

2. Возможность блокировки части мест из продажи на рейс.
6.«Бонусная программа» для пассажиров – накопление миль, использование их при оплате билетов.
9. Билеты с багажом и без багажа. Доплата за багаж.

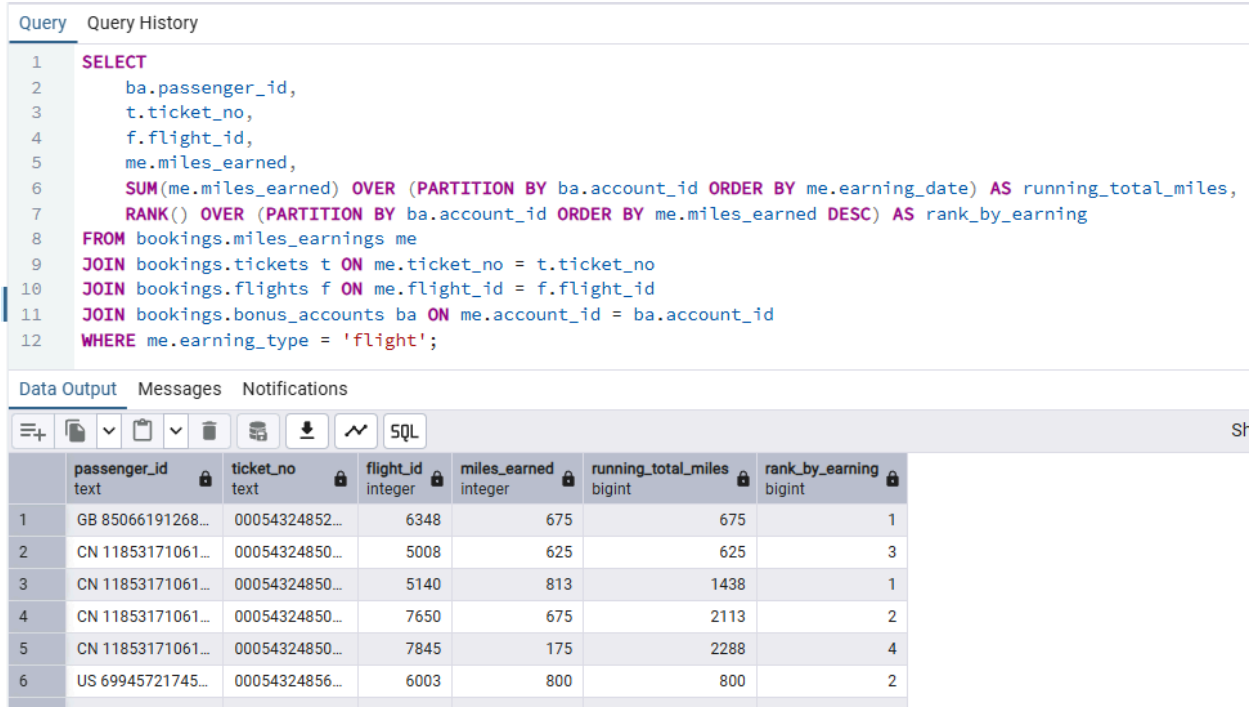
1. Заблокированные места на рейс с указанием процента блокировки от всех мест самолёта




```
SELECT
    b.flight_id,
    f.route_no,
    COUNT(b.seat_no) AS blocked_seats_count,
    (SELECT COUNT(*) FROM seats s WHERE s.airplane_code = a.airplane_code) AS
total_seats_on_plane,
    ROUND(100.0 * COUNT(b.seat_no) / NULLIF((SELECT COUNT(*) FROM seats s WHERE
s.airplane_code = a.airplane_code), 0), 2) AS blocked_percent
FROM bookings.blocked_seats b
JOIN bookings.flights f ON b.flight_id = f.flight_id
JOIN bookings.airplanes_data a ON a.airplane_code = (SELECT airplane_code FROM
```

```
bookings.seats LIMIT 1)
GROUP BY b.flight_id, f.route_no, a.airplane_code
HAVING COUNT(b.seat_no) > 0;
```

2. Накопление миль за перелёты



Query | Query History

```
SELECT
    ba.passenger_id,
    t.ticket_no,
    f.flight_id,
    me.miles_earned,
    SUM(me.miles_earned) OVER (PARTITION BY ba.account_id ORDER BY me.earning_date) AS running_total_miles,
    RANK() OVER (PARTITION BY ba.account_id ORDER BY me.miles_earned DESC) AS rank_by_earning
FROM bookings.miles_earnings me
JOIN bookings.tickets t ON me.ticket_no = t.ticket_no
JOIN bookings.flights f ON me.flight_id = f.flight_id
JOIN bookings.bonus_accounts ba ON me.account_id = ba.account_id
WHERE me.earning_type = 'flight';
```

Data Output | Messages | Notifications

	passenger_id text	ticket_no text	flight_id integer	miles_earned integer	running_total_miles bigint	rank_by_earning bigint
1	GB 85066191268...	00054324852...	6348	675	675	1
2	CN 11853171061...	00054324850...	5008	625	625	3
3	CN 11853171061...	00054324850...	5140	813	1438	1
4	CN 11853171061...	00054324850...	7650	675	2113	2
5	CN 11853171061...	00054324850...	7845	175	2288	4
6	US 69945721745...	00054324856...	6003	800	800	2

```
SELECT
    ba.passenger_id,
    t.ticket_no,
    f.flight_id,
    me.miles_earned,
    SUM(me.miles_earned) OVER (PARTITION BY ba.account_id ORDER BY
me.earning_date) AS running_total_miles,
    RANK() OVER (PARTITION BY ba.account_id ORDER BY me.miles_earned DESC) AS
rank_by_earning
FROM bookings.miles_earnings me
JOIN bookings.tickets t ON me.ticket_no = t.ticket_no
JOIN bookings.flights f ON me.flight_id = f.flight_id
JOIN bookings.bonus_accounts ba ON me.account_id = ba.account_id
WHERE me.earning_type = 'flight';
```

3. Использование миль при оплате билетов

Query | Query History

```
SELECT
    mb.burn_id,
    ba.passenger_id,
    t.ticket_no,
    t.book_ref,
    mb.miles_burned,
    mb.discount_amount,
    b.total_amount AS original_price,
    b.total_amount - mb.discount_amount AS final_price
FROM bookings.miles_burns mb
JOIN bookings.bonus_accounts ba ON mb.account_id = ba.account_id
JOIN bookings.tickets t ON mb.ticket_no = t.ticket_no
JOIN bookings.bookings b ON t.book_ref = b.book_ref
WHERE mb.discount_amount > 0;
```

Data Output | Messages | Notifications

	burn_id integer	passenger_id text	ticket_no text	book_ref character (6)	miles_burned integer	discount_amount numeric (10,2)	original_price numeric (10,2)	final_price numeric
1	81	CZ 48098741583...	00054324850...	9WLT74	125	500.00	10000.00	9500.00
2	82	AT 54002088862...	00054324846...	AV3TZ6	250	1000.00	25450.00	24450.00
3	83	CN 59944233415...	00054324849...	1WC485	88	350.00	3500.00	3150.00
4	84	CN 97138255841...	00054324861...	NIZA9L	300	1200.00	33000.00	31800.00
5	85	CN 70588362049...	00054324855...	LFC4AU	275	1100.00	11000.00	9900.00


```
SELECT
    mb.burn_id,
    ba.passenger_id,
    t.ticket_no,
    t.book_ref,
    mb.miles_burned,
    mb.discount_amount,
    b.total_amount AS original_price,
    b.total_amount - mb.discount_amount AS final_price
FROM bookings.miles_burns mb
JOIN bookings.bonus_accounts ba ON mb.account_id = ba.account_id
JOIN bookings.tickets t ON mb.ticket_no = t.ticket_no
JOIN bookings.bookings b ON t.book_ref = b.book_ref
WHERE mb.discount_amount > 0;
```

4. Билеты с багажом и без багажа

Query | Query History

```
SELECT
    t.book_ref,
    t.passenger_id,
    t.baggage_included,
    COUNT(DISTINCT t.ticket_no) AS tickets_count,
    SUM(t.baggage_weight_kg) AS total_kg,
    SUM(t.baggage_pieces) AS total_pieces,
    SUM(t.baggage_price) AS total_baggage_price
FROM bookings.tickets t
LEFT JOIN bookings.baggage b ON t.ticket_no = b.ticket_no AND t.baggage_included = true
GROUP BY t.book_ref, t.passenger_id, t.baggage_included;
```

Data Output | Messages | Notifications

	book_ref character (6)	passenger_id text	baggage_included boolean	tickets_count bigint	total_kg bigint	total_pieces bigint	total_baggage_price numeric
1	0000EG	CA 19852815975...	false	2	0	0	0.00
2	00027P	US 31246178696...	false	1	0	0	0.00
3	00027P	US 32917921601...	false	1	0	0	0.00
4	00040W	IN 2849974177127	false	2	0	0	0.00
5	000562	CN 81240861306...	false	2	0	0	0.00
6	000562	CN 82787803868...	false	2	0	0	0.00

```
SELECT
    t.book_ref,
    t.passenger_id,
    t.baggage_included,
    COUNT(DISTINCT t.ticket_no) AS tickets_count,
    SUM(t.baggage_weight_kg) AS total_kg,
    SUM(t.baggage_pieces) AS total_pieces,
    SUM(t.baggage_price) AS total_baggage_price
FROM bookings.tickets t
LEFT JOIN bookings.baggage b ON t.ticket_no = b.ticket_no AND t.baggage_included = true
GROUP BY t.book_ref, t.passenger_id, t.baggage_included;
```

5. Доплата за багаж

Query | Query History

```
SELECT
    t.ticket_no,
    t.passenger_name,
    b.baggage_weight_kg,
    b.baggage_pieces,
    (SELECT price_per_kg FROM bookings.baggage_rules br
     WHERE br.route_no = f.route_no
       AND br.fare_conditions = s.fare_conditions
     LIMIT 1) AS price_per_kg_rule,
    b.baggage_weight_kg * (SELECT price_per_kg FROM bookings.baggage_rules br LIMIT 1) AS calculated_extra_charge
FROM bookings.tickets t
JOIN bookings.segments s ON t.ticket_no = s.ticket_no
JOIN bookings.flights f ON s.flight_id = f.flight_id
JOIN bookings.baggage b ON t.ticket_no = b.ticket_no
WHERE t.baggage_included = false;
```

Data Output | Messages | Notifications

Showing rc

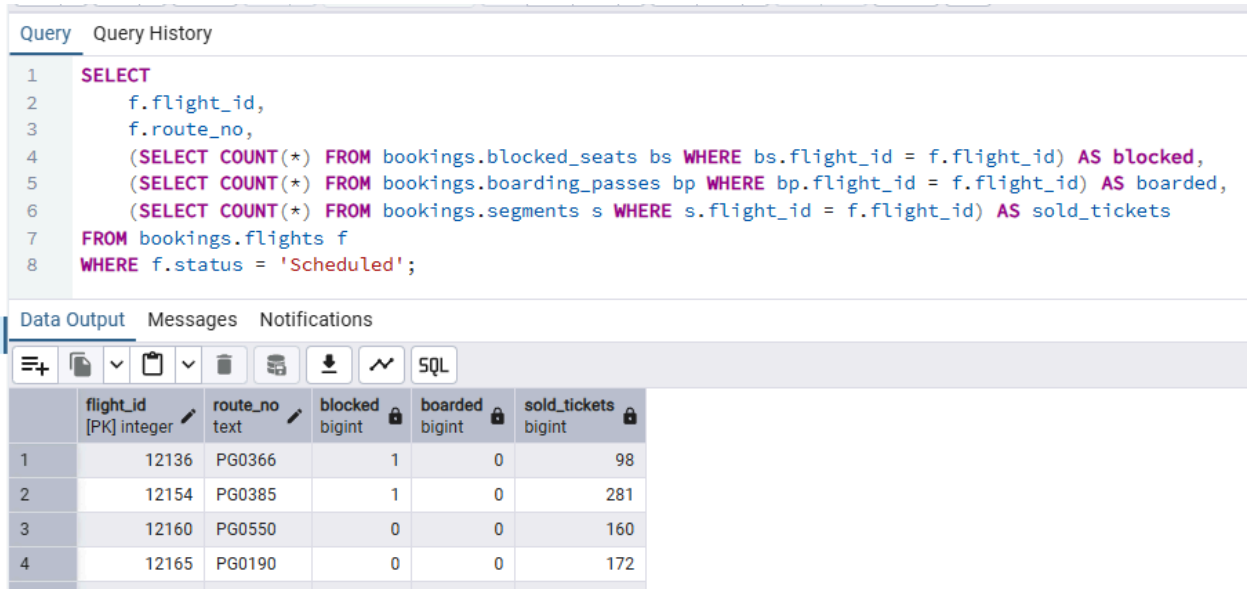
	ticket_no text	passenger_name text	baggage_weight_kg numeric (8,2)	baggage_pieces integer	price_per_kg_rule numeric (10,2)	calculated_extra_charge numeric
1	T001	Иван Петров	25.00	2	15.00	250.0000

```
SELECT
    t.ticket_no,
    t.passenger_name,
    b.baggage_weight_kg,
    b.baggage_pieces,
    (SELECT price_per_kg FROM bookings.baggage_rules br
     WHERE br.route_no = f.route_no
       AND br.fare_conditions = s.fare_conditions
     LIMIT 1) AS price_per_kg_rule,
    b.baggage_weight_kg * (SELECT price_per_kg FROM bookings.baggage_rules br LIMIT
1) AS calculated_extra_charge
FROM bookings.tickets t
JOIN bookings.segments s ON t.ticket_no = s.ticket_no
JOIN bookings.flights f ON s.flight_id = f.flight_id
JOIN bookings.baggage b ON t.ticket_no = b.ticket_no
WHERE t.baggage_included = false;
```

6. Сравнение накопленных и потраченных миль


```
SELECT
    ba.passenger_id,
    ba.miles_balance,
    SUM(me.miles_earned) AS total_earned,
    SUM(mb.miles_burned) AS total_burned,
    SUM(me.miles_earned) - SUM(mb.miles_burned) AS diff,
    SUM(SUM(me.miles_earned)) OVER (PARTITION BY ba.passenger_id) AS
running_earned
FROM bookings.bonus_accounts ba
LEFT JOIN bookings.miles_earnings me ON ba.account_id = me.account_id
LEFT JOIN bookings.miles_burns mb ON ba.account_id = mb.account_id
GROUP BY ba.passenger_id, ba.miles_balance;
```

7. Заблокированные места



	flight_id [PK] integer	route_no text	blocked bigint	boarded bigint	sold_tickets bigint
1	12136	PG0366	1	0	98
2	12154	PG0385	1	0	281
3	12160	PG0550	0	0	160
4	12165	PG0190	0	0	172

```
SELECT
    f.flight_id,
    f.route_no,
    (SELECT COUNT(*) FROM bookings.blocked_seats bs WHERE bs.flight_id = f.flight_id)
AS blocked,
    (SELECT COUNT(*) FROM bookings.boarding_passes bp WHERE bp.flight_id = f.flight_id)
AS boarded,
    (SELECT COUNT(*) FROM bookings.segments s WHERE s.flight_id = f.flight_id) AS
sold_tickets
FROM bookings.flights f
WHERE f.status = 'Scheduled';
```

8. Пассажиры с самым большим количеством миль

Query | Query History

```
1  SELECT DISTINCT
2      ba.passenger_id,
3      ba.miles_balance,
4      DENSE_RANK() OVER (ORDER BY ba.miles_balance DESC) AS balance_rank,
5      COUNT(t.ticket_no) OVER (PARTITION BY ba.passenger_id) AS total_tickets
6  FROM bookings.bonus_accounts ba
7  JOIN bookings.miles_earnings me ON ba.account_id = me.account_id
8  JOIN bookings.tickets t ON me.ticket_no = t.ticket_no
9  ORDER BY balance_rank
10 LIMIT 10;
```

Data Output | Messages | Notifications

	passenger_id text	miles_balance integer	balance_rank bigint	total_tickets bigint
1	CN 97138255841...	2700	1	5
2	IN 67079581333...	2700	1	4
3	CN 11853171061...	2288	2	4

```
SELECT DISTINCT
    ba.passenger_id,
    ba.miles_balance,
    DENSE_RANK() OVER (ORDER BY ba.miles_balance DESC) AS balance_rank,
    COUNT(t.ticket_no) OVER (PARTITION BY ba.passenger_id) AS total_tickets
FROM bookings.bonus_accounts ba
JOIN bookings.miles_earnings me ON ba.account_id = me.account_id
JOIN bookings.tickets t ON me.ticket_no = t.ticket_no
ORDER BY balance_rank
LIMIT 10;
```

9. Багаж, оплаченный отдельно (без багажа в тарифе)

Query | Query History

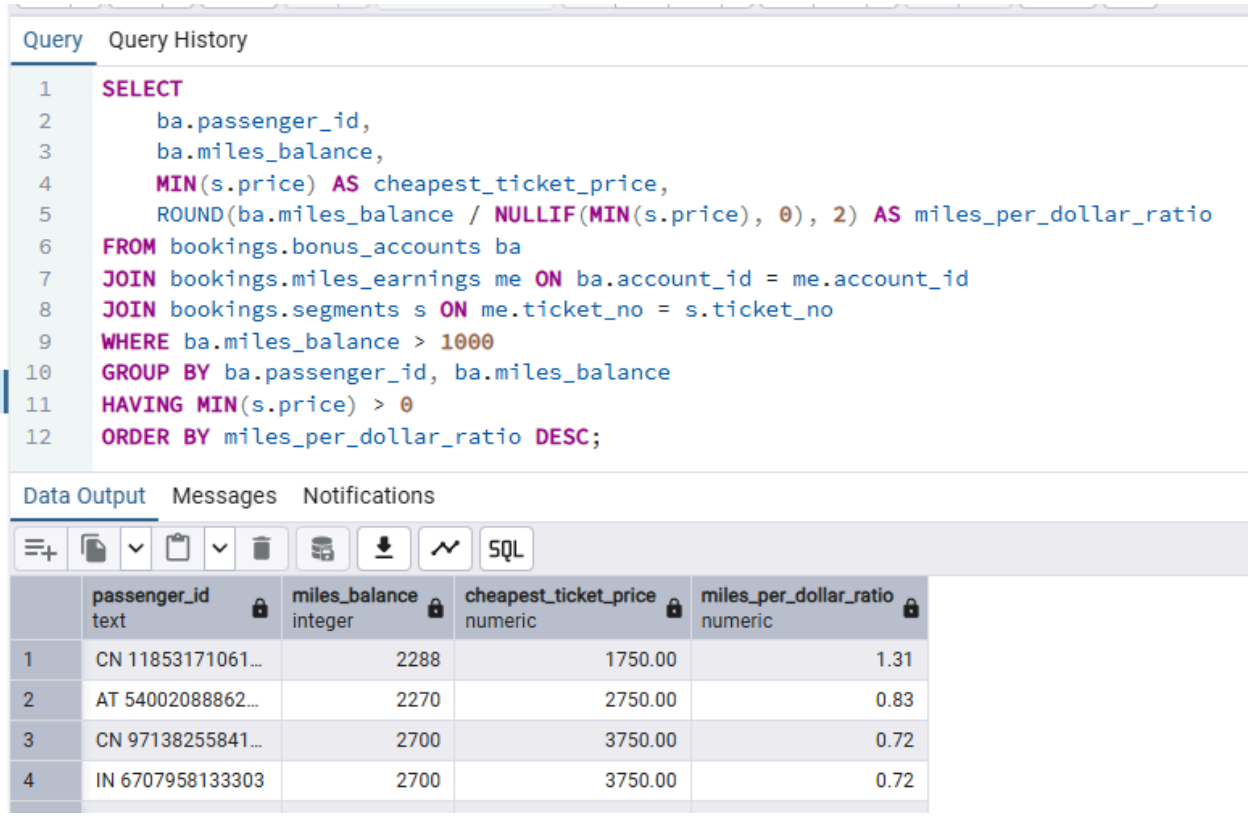
```
SELECT
    t.ticket_no,
    t.passenger_name,
    t.baggage_included,
    COALESCE(pb.total_kg, 0) AS actual_baggage_kg,
    t.baggage_price AS paid_extra,
    t.baggage_pieces
FROM bookings.tickets t
LEFT JOIN (
    SELECT ticket_no, SUM(baggage_weight_kg) AS total_kg
    FROM bookings.baggage
    GROUP BY ticket_no
) pb ON t.ticket_no = pb.ticket_no
WHERE t.baggage_price > 0 OR pb.total_kg > 0;
```

Data Output | Messages | Notifications

	ticket_no [PK] text	passenger_name text	baggage_included boolean	actual_baggage_kg numeric	paid_extra numeric (10,2)	baggage_pieces integer
1	T90101	Анна Смирнова	true	20.00	0.00	1
2	T90202	Олег Иванов	false	25.00	75.00	2
3	T90404	Дмитрий Соколов	true	32.00	30.00	2
4	T001	Иван Петров	false	25.00	75.00	2


```
SELECT
    t.ticket_no,
    t.passenger_name,
    t.baggage_included,
    COALESCE(pb.total_kg, 0) AS actual_baggage_kg,
    t.baggage_price AS paid_extra,
    t.baggage_pieces
FROM bookings.tickets t
LEFT JOIN (
    SELECT ticket_no, SUM(baggage_weight_kg) AS total_kg
    FROM bookings.baggage
    GROUP BY ticket_no
) pb ON t.ticket_no = pb.ticket_no
WHERE t.baggage_price > 0 OR pb.total_kg > 0;
```

10. Пассажиры, которые могли бы оплатить билет милями



Query | Query History

```
1  SELECT
2      ba.passenger_id,
3      ba.miles_balance,
4      MIN(s.price) AS cheapest_ticket_price,
5      ROUND(ba.miles_balance / NULLIF(MIN(s.price), 0), 2) AS miles_per_dollar_ratio
6  FROM bookings.bonus_accounts ba
7  JOIN bookings.miles_earnings me ON ba.account_id = me.account_id
8  JOIN bookings.segments s ON me.ticket_no = s.ticket_no
9  WHERE ba.miles_balance > 1000
10 GROUP BY ba.passenger_id, ba.miles_balance
11 HAVING MIN(s.price) > 0
12 ORDER BY miles_per_dollar_ratio DESC;
```

Data Output | Messages | Notifications

SQL

	passenger_id text	miles_balance integer	cheapest_ticket_price numeric	miles_per_dollar_ratio numeric
1	CN 11853171061...	2288	1750.00	1.31
2	AT 54002088862...	2270	2750.00	0.83
3	CN 97138255841...	2700	3750.00	0.72
4	IN 6707958133303	2700	3750.00	0.72

```
SELECT
    ba.passenger_id,
    ba.miles_balance,
    MIN(s.price) AS cheapest_ticket_price,
    ROUND(ba.miles_balance / NULLIF(MIN(s.price), 0), 2) AS miles_per_dollar_ratio
FROM bookings.bonus_accounts ba
JOIN bookings.miles_earnings me ON ba.account_id = me.account_id
JOIN bookings.segments s ON me.ticket_no = s.ticket_no
WHERE ba.miles_balance > 1000
GROUP BY ba.passenger_id, ba.miles_balance
HAVING MIN(s.price) > 0
ORDER BY miles_per_dollar_ratio DESC;
```

Лабораторная работа 5

Лабораторная работа 5. Реализовать в БД процедуры (триггеры, если нужно)
1) реализующие работу с изменениями в структуре (из лабораторной работы 2)
2) по заданию для лабораторной работы 5

Задание ЛР5 (вариант 15):

5. Отображения динамики пассажиропотока по разным направлениям.

1. Триггер для автоматического начисления миль при посадке пассажира


```
CREATE OR REPLACE FUNCTION award_miles_on_boarding()
RETURNS TRIGGER AS $$
DECLARE
   v_account_id INTEGER;
   v_price NUMERIC(10,2);
   v_miles_earned INTEGER;
BEGIN
   SELECT ba.account_id, s.price INTO v_account_id, v_price
   FROM tickets t
   JOIN segments s ON t.ticket_no = s.ticket_no
   LEFT JOIN bonus_accounts ba ON t.passenger_id = ba.passenger_id
   WHERE t.ticket_no = NEW.ticket_no
     AND s.flight_id = NEW.flight_id
```

```
    LIMIT 1;

    IF v_account_id IS NULL THEN
        INSERT INTO bonus_accounts (passenger_id, miles_balance, miles_earned_total,
miles_burned_total)
        SELECT t.passenger_id, 0, 0, 0
        FROM tickets t
        WHERE t.ticket_no = NEW.ticket_no
        ON CONFLICT (passenger_id) DO NOTHING
        RETURNING account_id INTO v_account_id;
    END IF;

    IF v_account_id IS NOT NULL AND v_price IS NOT NULL THEN
        v_miles_earned := (v_price / 10)::int;

        INSERT INTO miles_earnings (account_id, flight_id, ticket_no, miles_earned, price_paid,
earning_type)
        VALUES (v_account_id, NEW.flight_id, NEW.ticket_no, v_miles_earned, v_price, 'flight');

        UPDATE bonus_accounts
        SET miles_balance = miles_balance + v_miles_earned,
            miles_earned_total = miles_earned_total + v_miles_earned,
            updated_at = CURRENT_TIMESTAMP
        WHERE account_id = v_account_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_award_miles_on_boarding ON boarding_passes;
CREATE TRIGGER trg_award_miles_on_boarding
    AFTER INSERT ON boarding_passes
    FOR EACH ROW
    EXECUTE FUNCTION award_miles_on_boarding();

INSERT INTO boarding_passes (ticket_no, flight_id, seat_no, boarding_no, boarding_time)
VALUES ('0005432000000', 108, '12A', 999, CURRENT_TIMESTAMP)
ON CONFLICT DO NOTHING;
```

2. Триггер для проверки доступности мест при продаже билета


```
CREATE OR REPLACE FUNCTION check_seat_availability()
RETURNS TRIGGER AS $$
BEGIN
   IF EXISTS (
      SELECT 1 FROM blocked_seats
      WHERE flight_id = NEW.flight_id
        AND seat_no = NEW.seat_no
        AND is_active = TRUE
        AND (blocked_until IS NULL OR blocked_until > CURRENT_TIMESTAMP)
   ) THEN
      RAISE EXCEPTION 'Место % на рейс % заблокировано и не может быть продано',
NEW.seat_no, NEW.flight_id;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_check_seat_availability ON boarding_passes;
CREATE TRIGGER trg_check_seat_availability
   BEFORE INSERT ON boarding_passes
   FOR EACH ROW
   EXECUTE FUNCTION check_seat_availability();
```

3. Процедура блокировки места


```
CREATE OR REPLACE PROCEDURE block_seat(
    p_flight_id INTEGER,
    p_seat_no TEXT,
    p_reason TEXT,
    p_days_blocked INTEGER DEFAULT NULL
)
LANGUAGE plpgsql AS $$
DECLARE
    v_blocked_until TIMESTAMP WITH TIME ZONE;
BEGIN
    IF p_days_blocked IS NOT NULL THEN
        v_blocked_until := CURRENT_TIMESTAMP + (p_days_blocked || ' days')::interval;
    ELSE
        v_blocked_until := NULL;
    END IF;

    INSERT INTO blocked_seats (flight_id, seat_no, blocked_reason, blocked_by,
blocked_until, is_active)
    VALUES (p_flight_id, p_seat_no, p_reason, 'procedure', v_blocked_until, TRUE)
    ON CONFLICT (flight_id, seat_no)
    DO UPDATE SET
        blocked_reason = EXCLUDED.blocked_reason,
        blocked_until = EXCLUDED.blocked_until,
        is_active = TRUE,
        blocked_by = 'procedure';

    RAISE NOTICE 'Место % на рейс % заблокировано', p_seat_no, p_flight_id;
END;
$$;
```

4. Процедура для отображения динамики пассажиропотока


```
CREATE OR REPLACE FUNCTION get_passenger_traffic_dynamics(
    p_start_date DATE,
    p_end_date DATE,
    p_interval_type TEXT DEFAULT 'day'  -- 'day', 'week', 'month'
)
RETURNS TABLE (
    period_start DATE,
    period_end DATE,
    route TEXT,
    passengers_count BIGINT,
    revenue NUMERIC,
    running_total BIGINT
) AS $$
BEGIN
    RETURN QUERY
    WITH traffic AS (
        SELECT
            CASE p_interval_type
                WHEN 'day' THEN date_trunc('day', f.scheduled_departure)::date
                WHEN 'week' THEN date_trunc('week', f.scheduled_departure)::date
                WHEN 'month' THEN date_trunc('month', f.scheduled_departure)::date
                ELSE date_trunc('day', f.scheduled_departure)::date
```

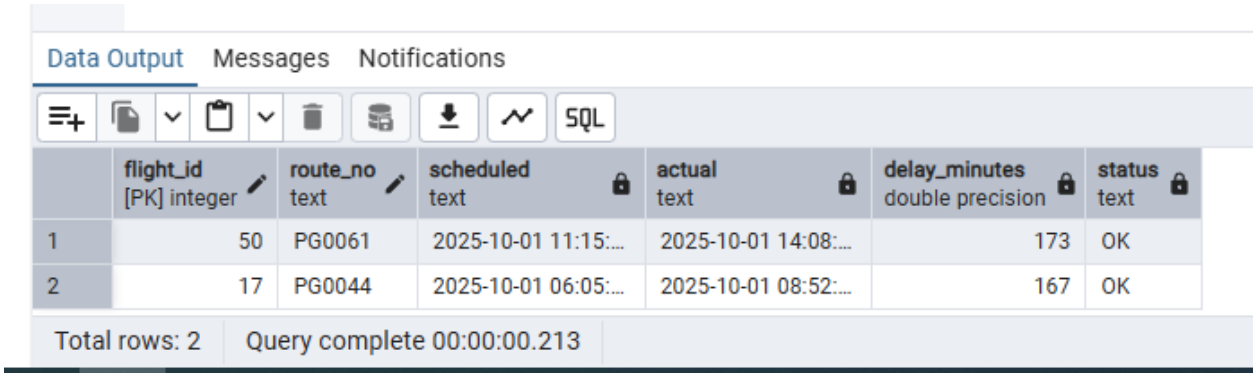
```
        END AS period,
        r.departure_airport || ' → ' || r.arrival_airport AS route_name,
        COUNT(DISTINCT t.ticket_no) AS passenger_count,
        SUM(s.price) AS total_revenue
    FROM flights f
    JOIN routes r ON f.route_no = r.route_no
    JOIN segments s ON f.flight_id = s.flight_id
    JOIN tickets t ON s.ticket_no = t.ticket_no
    WHERE f.scheduled_departure::date BETWEEN p_start_date AND p_end_date
    GROUP BY period, route_name
  )
  SELECT
    period AS period_start,
    period + (CASE p_interval_type
        WHEN 'day' THEN interval '1 day'
        WHEN 'week' THEN interval '7 days'
        WHEN 'month' THEN interval '1 month'
    END)::date - 1 AS period_end,
    route_name AS route,
    passenger_count,
    total_revenue,
    SUM(passenger_count) OVER (PARTITION BY route_name ORDER BY period) AS
running_total
  FROM traffic
  ORDER BY route_name, period;
END;
$$ LANGUAGE plpgsql;
```

Лабораторная работа 6

1. Какие рейсы вылетли из Внуково 1 октября 2025 с опозданием более чем 2 часа (ЛР1)

Было

```
SELECT DISTINCT
    f.flight_id,
    f.route_no,
    TO_CHAR(f.scheduled_departure, 'YYYY-MM-DD HH24:MI:SS') AS scheduled,
    TO_CHAR(f.actual_departure, 'YYYY-MM-DD HH24:MI:SS') AS actual,
    DATE_PART('hour', f.actual_departure - f.scheduled_departure) * 60 +
    DATE_PART('minute', f.actual_departure - f.scheduled_departure) AS delay_minutes,
    CASE
        WHEN DATE_PART('hour', f.actual_departure - f.scheduled_departure) > 2
        THEN 'OVERDUE'
        ELSE 'OK'
    END AS status
FROM bookings.flights f
JOIN bookings.routes r ON f.route_no = r.route_no
LEFT JOIN bookings.airports_data a ON r.arrival_airport = a.airport_code
WHERE LOWER(r.departure_airport) = 'vko'
  AND TO_CHAR(f.scheduled_departure, 'YYYY-MM-DD') = '2025-10-01'
  AND f.actual_departure IS NOT NULL
  AND (DATE_PART('hour', f.actual_departure - f.scheduled_departure) * 3600 +
       DATE_PART('minute', f.actual_departure - f.scheduled_departure) * 60 +
       DATE_PART('second', f.actual_departure - f.scheduled_departure)) / 3600 > 2
ORDER BY delay_minutes DESC;
```



Data Output | Messages | Notifications

	flight_id [PK] integer	route_no text	scheduled text	actual text	delay_minutes double precision	status text
1	50	PG0061	2025-10-01 11:15:...	2025-10-01 14:08:...	173	OK
2	17	PG0044	2025-10-01 06:05:...	2025-10-01 08:52:...	167	OK

Total rows: 2 | Query complete 00:00:00.213

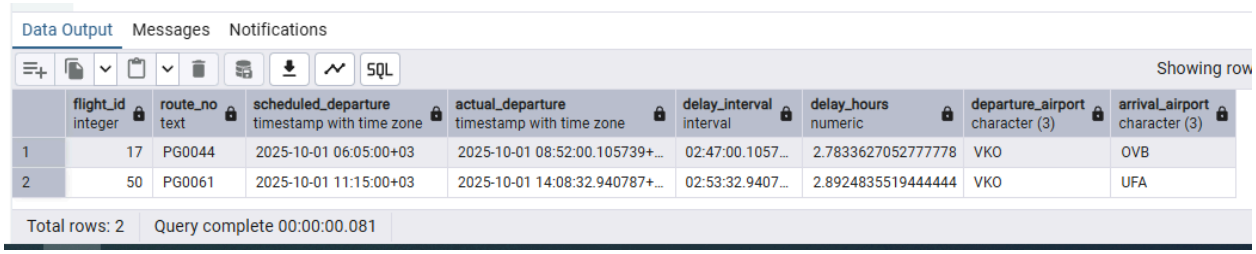
Стало

```
SELECT
    f.flight_id,
```

```
    f.route_no,
    f.scheduled_departure,
    f.actual_departure,
    (f.actual_departure - f.scheduled_departure) AS delay_interval,
    EXTRACT(EPOCH FROM (f.actual_departure - f.scheduled_departure)) / 3600 AS
delay_hours,
    r.departure_airport,
    r.arrival_airport
FROM bookings.flights f
JOIN bookings.routes r ON f.route_no = r.route_no
WHERE r.departure_airport = 'VKO'
  AND f.scheduled_departure >= '2025-10-01'
  AND f.scheduled_departure < '2025-10-02'
  AND f.actual_departure IS NOT NULL
  AND (f.actual_departure - f.scheduled_departure) > INTERVAL '2 hours'
ORDER BY f.scheduled_departure;
```

Добавить индексы

```
CREATE INDEX idx_flights_departure_date
ON bookings.flights(scheduled_departure);

CREATE INDEX idx_routes_departure
ON bookings.routes(departure_airport);
```



Время выполнения: ~0.2c -> 0.08c

2. Доплата за багаж (ЛР4)

Было

```
WITH passengers_with_5_tickets AS (
    SELECT passenger_id, passenger_name, COUNT(DISTINCT ticket_no) AS tickets_count
    FROM tickets
    GROUP BY passenger_id, passenger_name
    HAVING COUNT(DISTINCT ticket_no) = 5
),
passenger_tickets AS (
    SELECT pwt.passenger_id, pwt.passenger_name, t.ticket_no
    FROM passengers_with_5_tickets pwt
    JOIN tickets t ON pwt.passenger_id = t.passenger_id
),
ticket_segments AS (
    SELECT pt.passenger_id, pt.passenger_name, pt.ticket_no, s.flight_id,
           f.route_no, f.scheduled_departure, r.departure_airport, r.arrival_airport
    FROM passenger_tickets pt
    JOIN segments s ON pt.ticket_no = s.ticket_no
    JOIN flights f ON s.flight_id = f.flight_id
    JOIN routes r ON f.route_no = r.route_no
)
SELECT passenger_name, passenger_id, ticket_no,
       STRING_AGG(departure_airport || ' -> ' || arrival_airport ||
                  ' (' || TO_CHAR(scheduled_departure, 'DD.MM.YYYY HH24:MI') || ')',
                  '; ' ORDER BY scheduled_departure) AS route_with_dates
FROM ticket_segments
GROUP BY passenger_name, passenger_id, ticket_no
ORDER BY passenger_name, ticket_no;
```

Data Output | Messages | Notifications

	ticket_no text	passenger_name text	baggage_weight_kg numeric (8,2)	baggage_pieces integer	price_per_kg_rule numeric (10,2)	calculated_extra_charge numeric
1	T001	Иван Петров	25.00	2	15.00	250.0000

Total rows: 1 | Query complete 00:00:00.097

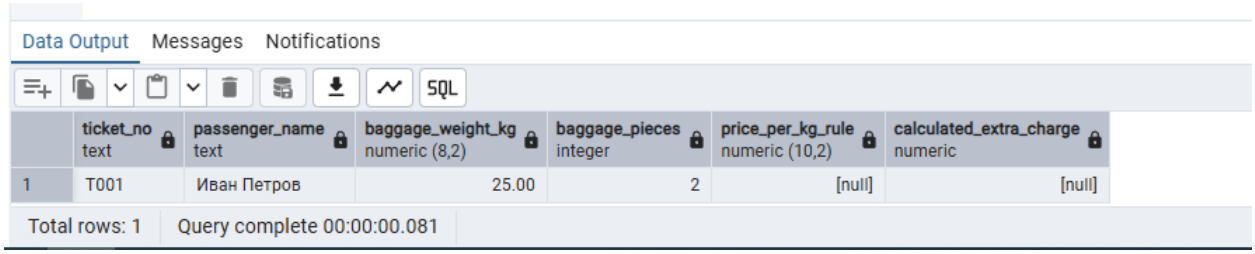
Индексы

```
CREATE INDEX idx_baggage_rules_route_fare
ON bookings.baggage_rules(route_no, fare_conditions)
INCLUDE (price_per_kg, price_per_piece, free_baggage_kg);

CREATE INDEX idx_baggage_rules_valid_dates
```

```
ON bookings.baggage_rules(valid_from, valid_to);
```

Стало

```
SELECT
    t.ticket_no,
    t.passenger_name,
    b.baggage_weight_kg,
    b.baggage_pieces,
    br.price_per_kg AS price_per_kg_rule,
    b.baggage_weight_kg * br.price_per_kg AS calculated_extra_charge
FROM bookings.tickets t
JOIN bookings.segments s ON t.ticket_no = s.ticket_no
JOIN bookings.flights f ON s.flight_id = f.flight_id
JOIN bookings.baggage b ON t.ticket_no = b.ticket_no
LEFT JOIN bookings.baggage_rules br
    ON br.route_no = f.route_no
    AND br.fare_conditions = s.fare_conditions
    AND br.valid_from <= CURRENT_DATE
    AND br.valid_to >= CURRENT_DATE
WHERE t.baggage_included = false
  AND b.baggage_weight_kg > 0;
```

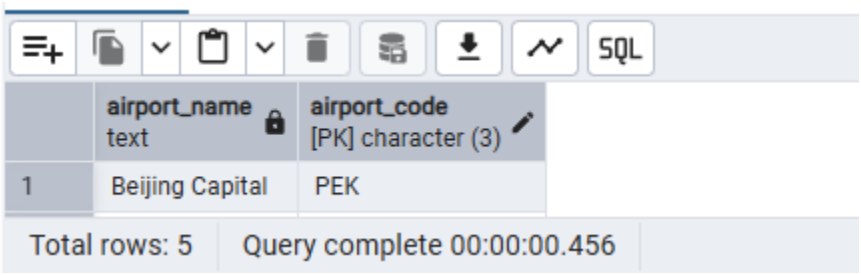


Время выполнения: ~0.1c -> 0.08c

3. Список аэропортов, в которых побывал пассажир по имени "Zhiming Hou" (ЛР1).

Было

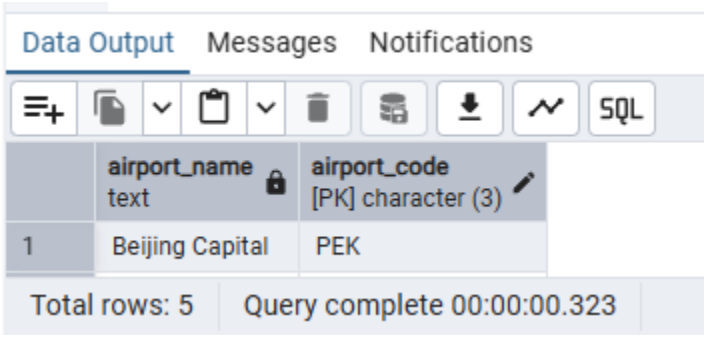
```
SELECT
    a.airport_name->>'en' as airport_name,
    a.airport_code
FROM tickets t
JOIN segments s ON t.ticket_no = s.ticket_no
JOIN flights f ON s.flight_id = f.flight_id
JOIN routes r ON f.route_no = r.route_no
JOIN airports_data a ON r.departure_airport = a.airport_code
WHERE t.passenger_name = 'Zhiming Hou'

UNION

SELECT
    a.airport_name->>'en' as airport_name,
    a.airport_code
FROM tickets t
JOIN segments s ON t.ticket_no = s.ticket_no
JOIN flights f ON s.flight_id = f.flight_id
JOIN routes r ON f.route_no = r.route_no
JOIN airports_data a ON r.arrival_airport = a.airport_code
WHERE t.passenger_name = 'Zhiming Hou';
```



Стало

```
SELECT DISTINCT
    a.airport_name->>'en' as airport_name,
    a.airport_code
FROM tickets t
JOIN segments s ON t.ticket_no = s.ticket_no
JOIN flights f ON s.flight_id = f.flight_id
JOIN routes r ON f.route_no = r.route_no
JOIN airports_data a ON r.departure_airport = a.airport_code
```

```
    OR r.arrival_airport = a.airport_code
WHERE t.passenger_name = 'Zhiming Hou';
```



Время выполнения: ~0.45c -> 0.3c

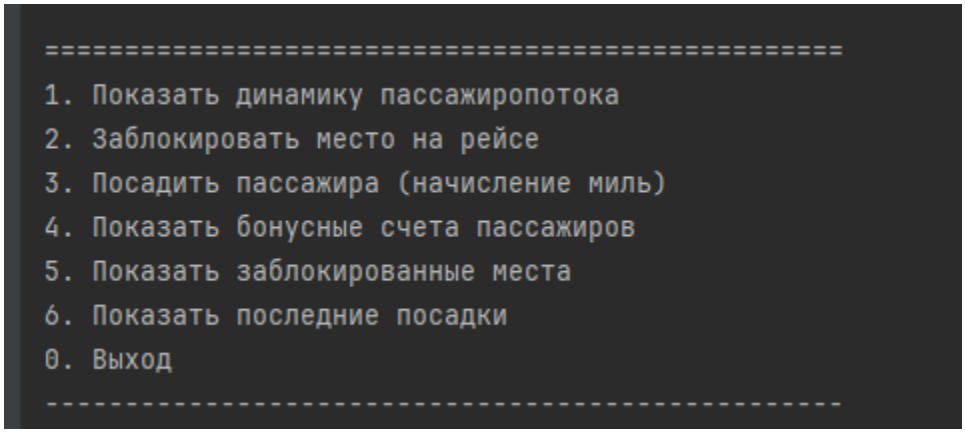
Лабораторная работа 7-8

Лабораторная работа 7. Реализовать программу для работы с БД, использующую ранее подготовленные процедуры.

Результат: программа на python, которая позволяет консольно вызывать процедуры.
К сожалению, так как я сильно опоздала со сдачей работ, у меня не было возможности спросить, как лучше сделать задание к этой работе, поэтому программа очень маленькая.

Библиотеки: psycopg2 (подключение к PostgreSQL), python-dotenv (конфигурация).

Файл с кодом лежит в репозитории. К отчёту приложены примеры работы каждого из пунктов:



1. Показать динамику пассажиропотока

Вызов функции get_passenger_traffic_dynamics()

Ручной ввод периода (ГГГГ-ММ-ДД)

Выбор интервала: день/неделя/месяц

Вывод: период, маршрут, кол-во пассажиров, выручка, накопленный итог

```
ДИНАМИКА ПАССАЖИРОПОТОКА
----------------------------------------
Введите даты в формате ГГГГ-ММ-ДД (например, 2026-01-01)
Или нажмите Enter для использования текущей даты

Дата начала (Enter - 30 дней назад): 2025-09-09
Дата окончания (Enter - сегодня): 2025-10-10

Период: 2025-09-09 -> 2025-10-10
Выберите интервал группировки:
1. День
2. Неделя
3. Месяц
Ваш выбор (1-3, Enter - день): 2

РЕЗУЛЬТАТЫ:
------------------------------------------------------------

Маршрут: AER -> DME
----------------------------------------
  2025-09-29 -> 2025-10-05 | Пассажиров: 234 | Выручка: 5292000.00 руб | Накоплено: 234
  2025-10-06 -> 2025-10-12 | Пассажиров: 156 | Выручка: 3528000.00 руб | Накоплено: 390

Маршрут: AER -> FRA
----------------------------------------
  2025-10-06 -> 2025-10-12 | Пассажиров: 65 | Выручка: 821250.00 руб | Накоплено: 65

Маршрут: AER -> LED
----------------------------------------
  2025-09-29 -> 2025-10-05 | Пассажиров: 78 | Выручка: 798000.00 руб | Накоплено: 78
  2025-10-06 -> 2025-10-12 | Пассажиров: 156 | Выручка: 1596000.00 руб | Накоплено: 234
```

2. Заблокировать место на рейсе

Вызов процедуры block_seat()

Ввод: ID рейса, номер места, причина, срок блокировки

Запись в таблицу blocked_seats

```
БЛОКИРОВКА МЕСТА
----------------------------------------
Введите ID рейса: 26
Введите номер места (например, 12A): 12A
Причина блокировки: maintenance
На сколько дней заблокировать (Enter - бессрочно): 1
Место 12A на рейс 26 успешно заблокировано
Нажмите Enter для продолжения...
```

+ запись появляется в заблокированных местах

```
ЗАБЛОКИРОВАННЫЕ МЕСТА
----------------------------------------
Рейс 26, Место 12A
  Причина: maintenance | Кем: procedure
  Заблокировано: 2026-06-16 19:51:15.564731+03:00
  Активно до: 2026-06-17 19:51:15.564731+03:00
----------------------------------------
```

3. Посадить пассажира (начисление миль)

Вставка в boarding_passes

Автоматическое срабатывание триггеров:

check_seat_availability - проверка блокировки

award_miles_on_boarding - начисление бонусных миль

```
----------------------------------------------------
Выберите действие (0-6): 3

ПОСАДКА ПАССАЖИРА
----------------------------------------
Номер билета: 0005432116149
ID рейса: 397
Номер места: 23C
Номер посадки: 1
Пассажир уже был посажен на этот рейс
Нажмите Enter для продолжения...
```

```
ПОСАДКА ПАССАЖИРА
----------------------------------------
Номер билета: 0005432816607
ID рейса: 72
Номер места: 12C
Номер посадки: 12345

Пассажир успешно посажен!
  Билет: 0005432816607, Рейс: 72, Место: 12C
  Триггер check_seat_availability: место проверено
  Триггер award_miles_on_boarding: мили начислены
  Начислено миль: 500 (за оплату 5000.00 руб)
Нажмите Enter для продолжения...
```

4. Показать бонусные счета пассажиров

```
------------------------------------------------
Выберите действие (0-6): 4

БОНУСНЫЕ СЧЕТА ПАССАЖИРОВ
--------------------------------------
ID: 130 | Пассажир: US 9329058187999
  Баланс миль: 39401 | Всего заработано: 37161 | Потрачено: 11325
  Создан: 2026-04-18 13:30:10.425712+03:00 | Обновлен: 2026-04-18 13:30:10.425712+03:00
--------------------------------------
ID: 124 | Пассажир: US 4302651646862
  Баланс миль: 39248 | Всего заработано: 22353 | Потрачено: 9344
  Создан: 2026-04-18 13:30:10.425712+03:00 | Обновлен: 2026-04-18 13:30:10.425712+03:00
--------------------------------------
ID: 135 | Пассажир: CN 5583054179010
  Баланс миль: 38372 | Всего заработано: 46105 | Потрачено: 3225
  Создан: 2026-04-18 13:30:10.425712+03:00 | Обновлен: 2026-04-18 13:30:10.425712+03:00
--------------------------------------
```

5. Показать заблокированные места

```
------------------------------------------------
Выберите действие (0-6): 5

ЗАБЛОКИРОВАННЫЕ МЕСТА
----------------------------------------
Рейс 116, Место 1A
  Причина: maintenance | Кем: system
  Заблокировано: 2026-05-09 18:15:07.995238+03:00
  Бессрочная блокировка
----------------------------------------
Рейс 21, Место 1A
  Причина: maintenance | Кем: system
  Заблокировано: 2026-05-09 18:15:07.995238+03:00
  Бессрочная блокировка
----------------------------------------
```

```
ЗАБЛОКИРОВАННЫЕ МЕСТА
----------------------------------------
Рейс 26, Место 12A
  Причина: maintenance | Кем: procedure
  Заблокировано: 2026-06-16 19:51:15.564731+03:00
  Активно до: 2026-06-17 19:51:15.564731+03:00
----------------------------------------
```

6. Показать последние посадки

```
------------------------------------------------
Выберите действие (0-6): 6

ПОСЛЕДНИЕ ПОСАДКИ
----------------------------------------
Билет: 0005432288550 | Рейс: 936 | Место: 43A
  Время посадки: None
----------------------------------------
Билет: 0005432088302 | Рейс: 397 | Место: 21B
  Время посадки: None
----------------------------------------
Билет: 0005432150625 | Рейс: 163 | Место: 11D
  Время посадки: None
----------------------------------------
```

Лабораторная работа 8. Доработка структуры, запросов, процедур и программного обеспечения.

Я не сдавала отдельно работу номер 7, поэтому не могу выполнить 8 работу полноценно. Предполагаемое улучшение: хранить пароль в env, а не в коде.

```
1 usage
class DBApp:
    def __init__(self):
        try:
            self.conn = psycopg2.connect(
                host="localhost",
                port=5432,
                database="demo",
                user="postgres",
                password="pass"
            )
            self.cur = self.conn.cursor()
            self.cur.execute("SET search_path TO bookings, public;")
            self.cur.execute("SET client_encoding TO 'UTF8';")

```

```
1 usage
class DBApp:
    def __init__(self):
        try:
            self.conn = psycopg2.connect(
                host=os.getenv('DB_HOST', 'localhost'),
                port=os.getenv('DB_PORT', 5432),
                database=os.getenv('DB_NAME', 'demo'),
                user=os.getenv('DB_USER', 'postgres'),
                password=os.getenv('DB_PASSWORD')
            )

```